

Independent Bookstore Discovery and Clubs

Project Documentation — objectives, scope, architecture, data model, APIs, AI design and acceptance criteria

Document	01 · Project Documentation (v1.0)
Institution	VNR Vignana Jyothi Institute of Engineering and Technology, Hyderabad
Team size	10 members (see Document 02 · Work Division and Integration Plan)
Stack	ReactJS, Node.js, Express, Mongoose, MongoDB Atlas + Atlas Vector Search, Ollama, LangChain JS, Zod
Date	06 October 2026

Contents

1. Introduction and problem statement
2. Objectives
3. Scope
4. Users, roles and permissions
5. Technology stack
6. System architecture
7. Functional requirements by module
8. Data model (MongoDB collections)
9. API specification
10. AI design: discovery, spoiler-aware RAG and agent
11. Security requirements
12. Non-functional requirements
13. Demonstration scenario
14. Acceptance and test checklist
15. Repository structure and conventions
16. Glossary

1. Introduction and problem statement

We are building an online catalog and reading-club portal for an **independent bookstore**. Readers frequently remember a *theme, a mood or a plot detail* rather than an exact title, so a keyword search box is not enough. The portal must help them discover books from the store's curated catalog, check **real local stock**, reserve a copy for pickup, and take part in moderated reading clubs **without running into spoilers unexpectedly**.

Booksellers maintain inventory, descriptions and store policies. Readers reserve copies, build reading lists and join clubs while declaring how far they have read (chapter or page). Moderators run meetings and approve discussion material. An AI assistant supports discovery and answers questions — but only from **permitted** descriptions, excerpts and store documents. It must never pretend to know the full text of a copyrighted book when only a synopsis is available.

1.1 Problems we solve

Problem today	How our application solves it
Readers can't find a book they only half-remember	Semantic (vector) search over catalog descriptions, with an explanation of which attributes matched
Online listings don't reflect what is actually on the shelf	Live inventory per condition (new / used / unavailable) and reservation holds with pickup windows
Book clubs spoil endings for slower readers	Each member declares progress; AI answers and discussion material are filtered to that reading boundary
Picking the next club book is slow and ad hoc	An agent shortlists books that match theme, have enough copies and fit the group's reading time
Generic chatbots hallucinate plot details	Retrieval restricted to approved source text, citations shown, graceful "I don't have that" answers

2. Objectives

2.1 Primary objectives

1. Deliver a secure, role-based full-stack web application (React + Express + MongoDB Atlas) for readers, booksellers, club moderators and a store administrator.
2. Provide **semantic book discovery** that matches remembered themes or preferences to permitted catalog descriptions and explains the match without inventing plot.
3. Provide **spoiler-aware RAG**: answer store-policy questions and discuss approved excerpts strictly within the reader's declared reading boundary.
4. Provide a **purposeful agent workflow** (searchCatalog → checkStockAndLength → compareClubConstraints) that produces an evidence-backed shortlist for club meetings.
5. Guarantee correct inventory under concurrency: no over-reservation during stock races, and abandoned holds expire automatically.
6. Meet the shared security checks: verified email, hashed passwords, server-side sessions, CSRF protection, rate limiting and audited staff actions.

2.2 Learning objectives for the team

- Design a production-style REST API with validation (Zod) and authorization enforced on the server *and* inside AI tools.
- Use embeddings and MongoDB Atlas Vector Search with metadata pre-filters.
- Build a LangChain JS agent with typed tools and observable execution status.

- Practise team engineering: shared contracts, feature branches, code review, integration testing and a reproducible demo.

2.3 Success criteria

Measure	Target for the final demo
Demo scenario	Runs end-to-end on seeded data in under 5 minutes, repeatable on a fresh database
Stock race test	N parallel reservations for a title with k copies produce exactly k successful holds
Spoiler test set	0 answers that cite an excerpt beyond the reader's declared boundary
Unsupported plot questions	100% answered with an honest "not in approved sources" response, never invented
Security checks	All items in Section 11 pass the checklist in Section 14
Citations	Every AI answer that uses sources shows them

3. Scope

3.1 In scope

- Account lifecycle: signup with email verification, login, logout, password recovery.
- Catalog browsing (public), semantic discovery, book detail with availability summary.
- Reservations with pickup windows, collection confirmation, cancellation and automatic expiry.
- Reading lists (private by default), reading-progress tracking with visibility control, spoiler settings.
- Clubs: publish descriptions, join, membership, meetings, discussion rules and approved material.
- AI: semantic discovery, spoiler-aware book chat, reading agent for club book selection.
- Staff tooling: stock management with audit trail, staff-role administration.

3.2 Out of scope

- Online payments and shipping (pickup in store only).
- Storing or serving full copyrighted book text — only synopses, publisher-approved excerpts and store documents.
- Native mobile apps (the React app is responsive instead).
- Free-form public forums; discussion happens inside moderated clubs.

4. Users, roles and permissions

Role	How the role is obtained	Responsibilities
Visitor	Not logged in	Browse catalog, themes, availability summaries and published clubs
Reader	Verified email signup	Save preferences, reserve and manage own reservations, reading lists, join clubs, control progress visibility, set spoiler limits, use book chat and reading agent
Bookseller	Granted by administrator only	Maintain catalog records, stock (new / used / unavailable), pickup windows; confirm collection; cancel expired holds
Club moderator	Granted per club by administrator	Manage that club's membership, meeting plan, discussion rules and approved material; select the club book
Administrator	Seeded at setup	Grant/revoke staff roles, resolve catalog or conduct disputes, view audit logs

4.1 Permission matrix

Action	Visitor	Reader	Bookseller	Moderator	Admin
Browse books / clubs	✓	✓	✓	✓	✓
Reserve a copy	—	✓ (verified)	—	✓ (as reader)	—
View / edit own reading list & history	—	owner only	—	owner only	—
Join club, update progress	—	✓ (verified)	—	✓	—
Book chat / reading agent	—	✓ members	✓	✓	✓
Update stock	—	—	✓ (audited)	—	✓
Confirm collection	—	—	✓	—	✓
Create meetings, approve material	—	—	—	own club only	✓
Change staff roles	—	—	—	—	✓ (never self)

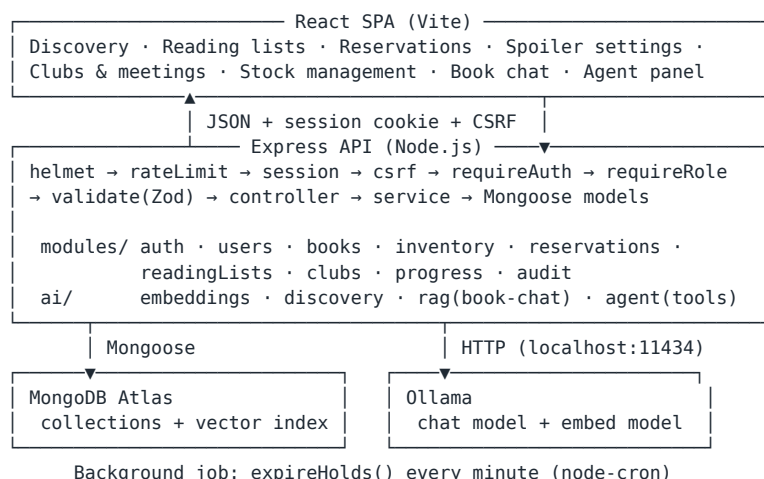
Golden rule: every rule above is enforced on the server **and** inside every AI tool. The React UI only hides buttons; it is never the security boundary. No user can ever assign a staff role to themselves.

5. Technology stack

Layer	Technology	Why we use it
Frontend	ReactJS (Vite), React Router, TanStack Query, Axios	Component-based SPA; query caching for catalog and reservation status
Backend	Node.js + Express	REST API, middleware chain for auth, CSRF, rate limits, validation
Validation	Zod	One schema per request/response shared by server, frontend and AI tool inputs
Database	MongoDB Atlas + Mongoose	Flexible documents; transactions and atomic updates for stock
Vector search	Atlas Vector Search	Semantic retrieval with metadata pre-filters (approved, spoiler position)
LLM & embeddings	Ollama (e.g. llama3.1:8b for chat, nomic-embed-text for embeddings)	Runs locally, free, reproducible demo
AI orchestration	LangChain JS	Retrievers, prompt templates, tool-calling agent
Sessions	express-session + connect-mongo	Expiring server-side sessions in httpOnly, secure, sameSite cookies
Security libs	argon2 / bcrypt, csrf-csrf, express-rate-limit, helmet	Password hashing, CSRF tokens, brute-force protection, headers
Email	Nodemailer + Ethereal / Mailtrap (dev)	Verification and password-reset links
Testing	Vitest/Jest, Supertest, mongodb-memory-server, Playwright	Unit, API, concurrency and end-to-end tests

6. System architecture

The system is a three-tier web application with an AI service layer living inside the Express server. All AI calls go through the same authentication and authorization middleware as normal routes.



6.1 Request lifecycle (example: reserve a copy)

1. React calls `POST /api/books/:id/reservations` with the session cookie and `x-csrf-token` header.
2. Middleware checks rate limit, session, CSRF token, that the user is logged in and email-verified.
3. Zod validates `{ condition, pickupWindowId }`.

4. Reservation service atomically decrements available stock (`findOneAndUpdate` with `available: { $gte: 1 }`) inside a transaction and creates the hold with an `expiresAt`.
5. Response returns the hold; React shows status. If stock was taken by a parallel request, the API returns 409 and the UI shows "no longer available".

7. Functional requirements by module

M1 Authentication & sessions

- Signup with email + password; account inactive for reservations/clubs until email verified via signed, expiring token.
- Login creates server-side session (rotated id); logout destroys it; idle and absolute expiry.
- Password recovery: single-use reset token (hashed in DB, 30 min expiry); invalidates other sessions.
- Login rate limit per IP + per email; generic error messages to avoid account enumeration.

M2 Users, roles & audit

- User profile with preferences (genres, moods, length) and spoiler settings.
- Administrator grants bookseller role and per-club moderator rights via `PATCH /api/users/:id/staff-role`.
- Self-assignment always rejected; every role change and stock change written to audit log.

M3 Catalog & inventory

- Book records: title, authors, themes, synopsis, page count, chapter count, estimated reading hours, catalog source and permission status.
- Inventory per book per condition: new / used, counts of total, available, held; explicit unavailable status.
- Public catalog browsing with filters and availability summary ("3 new available", "Unavailable — not reservable").

M4 Reservations

- Reader reserves an available copy; hold lasts until end of chosen pickup window.
- Reader can view and cancel own reservations; bookseller confirms collection.
- Abandoned holds expire automatically and stock is released; races never oversell.

M5 Reading lists & progress

- Create and manage reading lists (private by default, owner-only).
- `PATCH /api/reading-progress` records chapter/page per book (optionally per club) and visibility (private / club / public).
- Spoiler limit defaults to declared progress; reader may set a stricter limit.

M6 Clubs & meetings

- Public list of published clubs with description and meeting dates.
- Verified reader joins a club; moderator manages membership, rules and meeting plans.
- Moderator approves discussion material (excerpts with position metadata) and selects the club book.

M7 Semantic discovery (AI)

- Natural-language query → embedding → vector search over permitted descriptions.
- Result cards explain matching attributes (theme, mood, setting, length) drawn only from stored text.

M8 Spoiler-aware book chat (AI)

- POST /api/book-chat answers policy questions and discusses approved excerpts within reading boundary.
- Cites sources; refuses or asks for clarification when excerpt metadata cannot support the boundary.

M9 Reading agent (AI)

- POST /api/reading-agent runs searchCatalog → checkStockAndLength → compareClubConstraints.
- Streams tool execution status; returns shortlist with evidence; never reserves or selects on its own.

8. Data model (MongoDB collections)

All collections use timestamps. Field lists are the agreed contract; owners may add fields but must not rename these without a contract change (see Document 02).

Collection	Key fields	Indexes / notes
users	email, passwordHash, emailVerifiedAt, roles: ['reader' 'bookseller' 'admin'], moderatorOf: [clubId], preferences {genres, moods, maxPages}, spoilerSettings {mode, strict}	unique email
sessions	managed by connect-mongo	TTL on expires
tokens	userId, type: verify reset, tokenHash, expiresAt, usedAt	TTL on expiresAt
catalogSources	name, type: publisher store synopsis, license, permission: approved pending revoked	—
books	title, authors[], isbn, themes[], moods[], synopsis, pageCount, chapterCount, readingHours, sourceId, contentLevel: synopsis excerpts, embedding[768]	text index on title/authors; vector index on embedding
inventory	bookId, condition: new used, total, available, held, status: active unavailable, pickupWindows[]	unique (bookId, condition)
reservations	userId, bookId, condition, status: held collected cancelled expired, pickupWindow, expiresAt, collectedBy	index (status, expiresAt)
readingLists	ownerId, name, visibility: private public, items[{bookId, addedAt, note}]	index ownerId
clubs	name, description, published, rules, currentBookId, moderators[], readingPace {pagesPerWeek}	—
memberships	clubId, userId, status: active removed, joinedAt	unique (clubId, userId)
meetings	clubId, date, bookId, agenda, chapterRange {from, to}, createdBy	index clubId
progress	userId, bookId, clubId?, chapter, page, visibility: private club public	unique (userId, bookId, clubId)
excerpts	bookId, sourceId, text, chapter, pageStart, pageEnd, approved, approvedBy, embedding[768]	vector index with filter fields bookId, approved, chapter

Collection	Key fields	Indexes / notes
storeDocs	title, text, type: policy event faq, chunkIndex, embedding[768]	vector index
conversations	userId, kind: chat agent, messages[{role, content, citations[], toolEvents[]}]	owner-only
auditLogs	actorId, action, target {type,id}, before, after, at, ip	append-only

9. API specification

Base path /api. JSON in, JSON out. Every response uses the envelope { ok: true, data } or { ok: false, error: { code, message, details? } }. All state-changing requests require the CSRF header.

Method & route	Access	Purpose
POST /auth/signup	Public	Create account, send verification email
GET /auth/verify?token=	Public	Verify email
POST /auth/login · POST /auth/logout	Public / session	Start / end session (rate limited)
POST /auth/forgot · POST /auth/reset	Public	Password recovery
GET /auth/me · GET /auth/csrf	Session	Current user, fetch CSRF token
GET /books	Public	Browse catalog, themes, availability summaries (?q, ?theme, ?semantic=true)
GET /books/:id	Public	Book detail with availability
GET /clubs	Public	Published clubs and meeting dates
POST /books/:id/reservations	Reader (verified)	Reserve a copy
GET /reservations/mine · DELETE /reservations/:id	Reader (owner)	List / cancel own reservations
POST /reading-lists (+ GET/PATCH/DELETE)	Reader (owner)	Manage reading lists
POST /clubs/:id/join	Reader (verified)	Join a club
PATCH /reading-progress	Reader	Update progress and visibility
PATCH /books/:id/stock	Bookseller	Change stock (audited)
POST /reservations/:id/collection	Bookseller	Confirm pickup
POST /clubs/:id/meetings	Moderator of club	Create meeting
POST /clubs/:id/excerpts/:eid/approve	Moderator of club	Approve discussion material
POST /book-chat	Members	Spoiler-aware RAG answer with citations
POST /reading-agent	Members	Agent shortlist with tool status (SSE stream)
PATCH /users/:id/staff-role	Administrator	Grant/revoke bookseller or moderator role

9.1 Standard error codes

HTTP	code	When
400	VALIDATION_ERROR	Zod validation failed (details lists fields)
401	UNAUTHENTICATED	No / expired session
403	FORBIDDEN · EMAIL_NOT_VERIFIED · CSRF_INVALID	Role, ownership, verification or CSRF failure
404	NOT_FOUND	Resource missing or not visible to caller
409	OUT_OF_STOCK · CONFLICT	Stock race lost, duplicate membership
429	RATE_LIMITED	Too many attempts
503	AI_UNAVAILABLE	Ollama unreachable — UI shows graceful fallback

10. AI design

10.1 Semantic book discovery

Input: free-text memory ("a quiet book about a lighthouse keeper and grief") plus optional saved preferences. **Process:** embed the query with Ollama → `$vectorSearch` on `books.embedding` filtered to approved sources → top-k candidates → LLM writes a short *why it matched* note using only the stored themes, moods and synopsis. **Rule:** the explanation may only reference attributes present in the record; the prompt forbids inventing plot.

10.2 Spoiler-aware RAG (book chat)

1. Determine the reader's boundary = $\min(\text{declared progress, explicit spoiler limit})$ for that book (and club, if asked in a club).
2. Retrieve from excerpts with pre-filter `{ bookId, approved: true, chapter: { $lte: boundary } }` and from `storeDocs` for policy questions.
3. If the question concerns a book with `contentLevel: 'synopsis'` only, or excerpts lack chapter/page metadata, the assistant asks for clarification or declines the spoiler-sensitive part.
4. Generate the answer from retrieved chunks only; attach citations (source, chapter/page). If nothing relevant is retrieved, return the graceful missing-data response.
5. Post-check: reject any citation whose position exceeds the boundary (defence in depth).

Honesty rule: "I only have the publisher's synopsis for this book, so I can't tell you how the subplot resolves." is a correct answer. Guessing the ending is a failed test.

10.3 Purposeful agent workflow

Example request: "Find a short mystery for our next club meeting."

Step	Tool	Input → Output	Rules
1	<code>searchCatalog(preferences)</code>	<code>{themes, moods, maxPages}</code> → candidate books with match evidence	Approved descriptions only
2	<code>checkStockAndLength(bookIds, groupSize)</code>	→ books with available copies $\geq$ groupSize and length	Reads inventory; never reserves
3	<code>compareClubConstraints(clubId, books)</code>	→ time-fit and content-fit notes vs next meeting date & club pace	Caller must be member/moderator of club
4	Final answer	Shortlist (3 max) with evidence and citations	Moderator selects; readers reserve separately

Every tool input is a Zod schema, every tool re-checks the caller's permissions, and each tool call emits a status event (started / succeeded / failed) that the UI renders live.

11. Security requirements

- Passwords hashed with argon2id (or bcrypt cost ≥ 12); never logged.
- Server-side sessions in MongoDB; cookies httpOnly, secure, sameSite=lax; session id rotated at login; idle timeout 30 min, absolute 8 h.
- CSRF: double-submit token on all POST/PATCH/PUT/DELETE.
- Rate limits on login, signup, forgot-password and AI endpoints.
- Email verification required before reservations and club participation.
- Bookseller role granted only by admin; moderator rights per club; no self-assignment of staff roles.
- Reading histories, progress (unless shared) and private lists are owner-only — enforced in queries (`{ ownerId: req.user.id }`).
- AI tools run with the caller's identity and repeat all authorization checks; the LLM cannot widen access.
- Staff stock changes and role changes write an audit record (before / after values).
- A recommendation never reserves a copy; reservation requires an explicit reader action.
- Input validation with Zod on every route; helmet security headers; no secrets in the repository (.env).

12. Non-functional requirements

Area	Requirement
Performance	Catalog list < 500 ms on seeded data; first AI token < 5 s on a laptop with Ollama
Reliability	Hold-expiry job is idempotent; stock updates atomic
Usability	Responsive layout; clear distinction between Unavailable and Reservable; loading and error states everywhere
Accessibility	Semantic HTML, labels on all inputs, keyboard navigation, contrast AA
Observability	Structured logs (pino) with request id; AI tool events stored in conversations
Reproducibility	npm run seed creates the full demo dataset from permitted documents

13. Demonstration scenario

A popular title matches the theme but lacks enough copies; the agent finds an available alternative that fits the group's reading time.

1. Moderator of "Tuesday Mystery Circle" (8 members, next meeting in 14 days, pace ≈ 120 pages/week) opens the agent panel.
2. Asks: "Find a short mystery for our next club meeting."
3. UI shows `searchCatalog` running → returns 5 candidates; the popular seeded title ranks first.
4. `checkStockAndLength` shows the popular title has only 3 copies (< 8) → flagged "not enough copies".
5. `compareClubConstraints` finds an alternative (≈ 220 pages, 10 used + new copies) that fits 14 days at the club's pace.
6. Agent returns a cited shortlist; moderator selects the alternative; a reader separately reserves a copy.

7. Bonus: a reader at chapter 4 asks book-chat about the ending → assistant declines the spoiler and offers chapter 1–4 discussion.

14. Acceptance and test checklist

#	Check	How it is verified
A1	Stock race	Fire 20 parallel reservations for a title with 3 copies → exactly 3 succeed, 17 receive 409
A2	Hold expiry	Create hold with past expiresAt, run job → status expired, available restored
A3	Unavailable vs reservable	Unavailable titles show no Reserve button and API returns 409
A4	Spoiler boundary	Reader at ch. 3 asks about ch. 10 → no ch. 10 citation, safe refusal
A5	Missing metadata	Excerpt without chapter → assistant asks for clarification
A6	Unsupported plot question	Synopsis-only book, ask for ending → honest decline
A7	Citations	Every source-based answer includes citations rendered in UI
A8	Tool status	Agent panel shows each tool started/succeeded/failed
A9	Graceful failure	Stop Ollama → UI shows friendly AI_UNAVAILABLE message, rest of app works
A10	Role safety	Reader calls PATCH /users/:self/staff-role → 403; attempt is audited
A11	Owner-only data	User B cannot read user A's private list or progress (404)
A12	Auth security	CSRF missing → 403; 6 bad logins → 429; unverified user cannot reserve
A13	Audit	Stock change writes before/after audit record
A14	Recommendation ≠ reservation	Agent run creates zero reservations

15. Repository structure and conventions

```

bookstore-clubs/           (single GitHub repo, npm workspaces)
├─ shared/                 Zod schemas + constants used by server and client
│   ├── schemas/ auth.ts books.ts reservations.ts clubs.ts ai.ts ...
│   └── constants.ts       roles, statuses, error codes
├─ server/
│   ├── src/app.js server.js
│   ├── src/middleware/    auth, roles, csrf, ratelimit, validate, error
│   ├── src/modules/<module>/ model.js routes.js controller.js service.js *.test.js
│   ├── src/ai/            ollama.js embeddings.js discovery/ rag/ agent/
│   ├── src/jobs/expireHolds.js
│   └── scripts/seed.js data/ (permitted seed documents)
├─ client/
│   └── src/ api/ components/ features/<feature>/ pages/ routes.jsx
├─ docs/                   these PDFs, API notes, ADRs
└─ .github/workflows/ci.yml lint + tests on every PR

```

- Branches: main (protected) ← dev ← feat/<module>-<short-desc>.
- Commits: Conventional Commits (feat(reservations): atomic hold).
- Every PR: passing CI, one reviewer approval, linked task, no secrets.
- Code style: ESLint + Prettier shared config; async/await; services contain logic, controllers stay thin.

16. Glossary

Term	Meaning
Embedding	A list of numbers representing the meaning of text; similar meanings → nearby vectors
Vector search	Finding stored items whose embeddings are closest to the query embedding
RAG	Retrieval-Augmented Generation — retrieve relevant text first, then let the LLM answer only from it
Reading boundary	The furthest chapter/page the reader allows the assistant to use
Hold	A reservation that keeps a copy aside until the pickup window ends
Agent	An LLM that decides which typed tools to call, in which order, to complete a task
CSRF	Cross-Site Request Forgery — prevented by requiring a secret token header